APT: Adaptive Pruning and Tuning — Slide Deck

Auto-generated

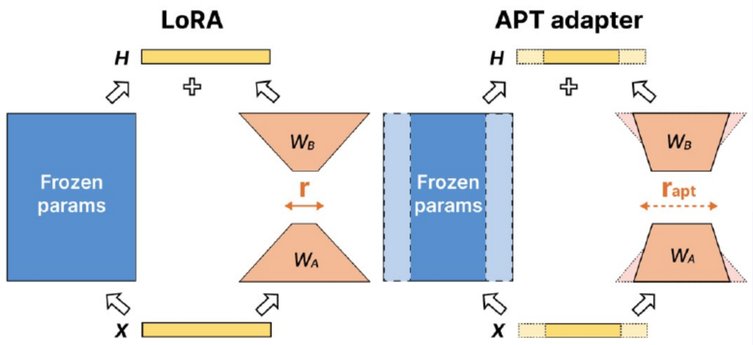
N/A • 2026-05-25

01 APT: Adaptive Pruning and Tuning — Paper and Authors

Model	Method	MNLI	SST2	SQuAD v2	CNN/DM	Train Time(⇓)	Train Mem(⇓)	Inf Time(⇓)	Inf Mem(⇓)
$\text{RoBERTa}_{\text{base}}$	FT	87.6	94.8	82.9	-	100.0%	100.0%	100.0%	100.0%
	LoRA	87.5	95.1	83.0	-	2137.0%	60.5%	100.0%	100.0%
	LoRA+Prune	84.0	93.0	79.2	-	5128.3%	**60.5%**	**38.0%**	**75.1%**
	Prune+Distill	**87.3**	**94.5**	-	-	1495.3%	168.5%	38.6%	79.2%
	LoRA+Prune+Distill	84.2	91.9	-	-	6534.6%	141.4%	39.4%	82.3%
	APT	86.4	**94.5**	**81.8**	-	**592.1%**	70.1%	41.3%	78.1%
T5_{base}	FT	87.1	95.2	-	42.1/20.3/39.4	100.0%	100.0%	100.0%	100.0%
	LoRA	87.0	95.0	-	38.7/17.2/36.0	255.5%	62.0%	100.0%	100.0%
	LoRA+Prune	80.9	92.3	-	36.7/15.7/33.9	4523.5%	**62.0%**	**47.1%**	**73.4%**
	APT	**87.0**	**95.0**	-	**38.6/17.0/35.8**	**484.7%**	73.9%	74.6%	81.5%

- Bowen Zhao, Hannaneh Hajishirzi, and Qingqing Cao present APT.
- APT unifies structured pruning and parameter-efficient tuning for efficient training and inference.
- The method targets high accuracy retention with real inference speed and memory gains.

02 Motivation: PEFT vs. Structured Pruning—A False Trade-off



- PEFT (parameter-efficient fine-tuning) reduces training memory but does not reduce inference cost (e.g., LoRA = Low-Rank Adaptation).
- Structured pruning improves inference efficiency but raises training cost.
- APT (Adaptive Pruning and Tuning) combines adaptive pruning with salience-guided tuning for faster training and real inference gains.

03 Practical Pain Points: Convergence, Memory, and Inference Cost

- Two-stage pipelines (fine-tuning (FT) then prune, or prune then retrain) slow convergence and cause unstable training.
- Dense optimizer states and teacher activations raise peak training memory.
- Adapters do not cut inference cost unless merged and often fail to reduce model size.
- Naive combinations suffer objective mismatch and misaligned static ranks.

04 Design Principles Behind APT

- Early pruning reduces activations and optimizer states to lower per-step cost.
- Salience-guided selection uses activation–gradient signals with outlier protection.
- Joint budgets track interim and target sparsity and tuning growth for deployable models.
- Zero-impact rank insertion and shared-parameter self-distillation improve stability.

05 Related Work: PEFT Landscape and Gaps

- Selective, additive, and dynamic PEFT methods enable efficient tuning.
- Prior methods keep fixed or shrinking tuning shapes without inference efficiency.
- APT adapts pruning and tuning jointly to improve training efficiency and inference speed.

06 Related Work: Compression and PEFT+Pruning Hybrids—Why They Break

- Quantization reduces memory but often needs runtime support for speedups.
- Structured pruning yields broad speedups but increases training cost and complexity.
- Prior hybrids lose accuracy under structured pruning or static tuning choices.
- APT selects pruning and tuning by salience during fine-tuning to minimize accuracy loss.

07 Motivation: Deployment Constraints and Pitfalls

- Single-GPU budgets (24–48 GB) are exceeded by dense optimizer states, masks, and teacher activations.
- Two-stage pipelines lengthen wall-clock and destabilize convergence.
- Adapters (e.g., LoRA) don't cut inference FLOPs/memory unless merged; naive combos still leave deployment cost high.
- Structured pruning helps inference but can erase useful representations without salience and demands costly recovery.
- Real deployment wins require **structured pruning guided by salience**, not naive pruning or static adapters.

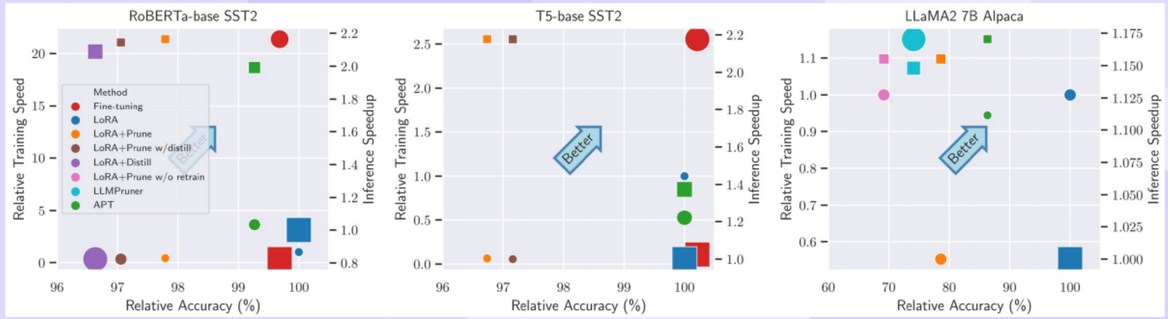
08 Real-world Failures We Observed

- Accuracy cliff after pruning LoRA-tuned models without alignment.
- Peak training memory spikes from masks, optimizer states, and teacher activations.
- Instability from static adapter ranks under changing sparsity.

09 What We Need from a Practical Method

- single-pass pipeline
- fits 24–48 GB without spikes
- real inference FLOP/memory cuts post-merge

10 Why Prior Combos Fail in Practice: Case Snapshots



- Post-pruning after LoRA (Low-Rank Adaptation) causes an accuracy cliff due to misaligned capacity.
- Pruning during fine-tuning triggers memory spikes from masks and optimizer states.
- Static adapter ranks under changing sparsity lead to instability and slow convergence.

11 Problem Setup: Joint Sparsity and Tuning Budgets

- The objective minimizes task loss under evolving sparsity and tuning budgets over training.
- The controls include pruning masks and adapter ranks that change over time.
- Higher final sparsity improves inference but risks accuracy loss, while early sparsity improves training efficiency. APT (Adaptive Pruning and Tuning) dynamically adjusts both.

12 APT Overview: Early Prune, Targeted Capacity Growth

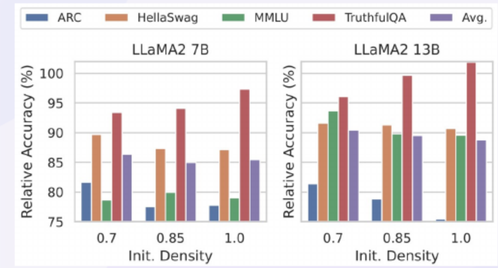
- Early structured pruning reduces training cost while preserving essential capacity.
- Salience-driven rank growth restores accuracy in the most impacted layers.
- The result is faster training and smaller inference models with high retention.

13 APT Adapter and Salience/Joint Search

- LoRA-style adapters use input/output masks with dynamic rank and are mergeable post-training for compact inference.
- Placed in attention projections (MHA) and, for smaller models, in FFN layers.
- Prunable targets: attention heads, hidden dimensions, and FFN neurons under kernel-friendly constraints.
- Preserve tensor shapes and head/dimension divisibility; keep masking consistent across W_A/W_B and attention outputs.

14 Adaptive Tuning: Rank Growth and Stability

- APT (Adaptive Pruning and Tuning) computes adapter importance by aggregating salience to target capacity growth effectively.
- Ranks are increased within budget for the most salient adapters to boost accuracy recovery.
- Gaussian rows in one matrix with zeros in the other preserve outputs at insertion.



15 Self-Distillation for Efficient Recovery

- Frozen shared parameters and duplicated tuning layers enable a lightweight teacher.
- A blended objective balances distillation with task learning over training.
- Block-wise teacher sampling and identity-initialized transformations stabilize learning.

16 Positioning APT: When To Use It (and When Not)

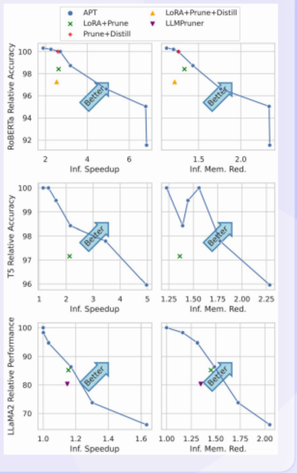
- APT fits tight GPU budgets that need training speed and inference efficiency.
- APT suits deployments that require a compact merged model with real speedups.
- Plain PEFT or specialized compression is preferable when inference cost is unchanged or extreme sparsity is required.

17 Evaluation Metrics: Definitions and Measurement Protocols

Method		$\mathcal{A}_P$	$\mathcal{A}_T$	Training T	Training M	Inference T	Inference M
PEFT	Adapter (Pfeiffer et al., 2021)	✗	✗	⇑High	⇓Low	⇑Low	⇑Low
	LoRA (Hu et al., 2022)	✗	✗	⇑High	⇓Low	=	=
	AdaLoRA (Zhang et al., 2023b)	✗	✓	⇑High	⇓Low	=	=
Pruning	MvP (Sanh et al., 2020)	✗	✗	⇑High	⇑Low	⇓Low	⇓Low
	BMP (Lagunas et al., 2021)	✗	✗	⇑High	⇑Low	⇓High	⇓Low
	CoFi (Xia et al., 2022)	✗	✗	⇑High	⇑Low	⇓High	⇓Low
	MT (Kwon et al., 2022)	✗	✗	=	=	⇓High	⇓Low
Combined	SPA (Hedegaard et al., 2022)	✗	✗	⇑High	⇑Low	⇓High	⇓Low
	LRP (Zhang et al., 2023a)	✗	✗	⇑High	⇓Low	⇓High	⇓Low
	APT (ours)	✓	✓	⇑Low	⇓Low	⇓High	⇓Low

- Accuracy retention = 100 × (APT score / dense fine-tuning (FT) score) — measured on the same data/splits.
- Accuracy retention, wall-clock, throughput, and peak training memory are reported consistently.
- GPU peak RSS = resident set size; includes KV (key–value) cache where applicable
- Identical tokenization, lengths, precision, and seeds ensure fair comparisons.
- Training wall-clock excludes validation/evaluation phases unless explicitly stated.
- Throughput = examples/sec averaged over steady-state steps after warm-up

18 Main Results: Accuracy Retention vs. Efficiency (RoBERTa/T**5**)



- APT retains up to 99 percent of FT (fine-tuning) accuracy with about 70 percent training memory at moderate sparsity.
- APT delivers about eight times faster pruning than LoRA with post pruning while improving accuracy at matched sparsity.
- APT achieves strong inference gains with competitive speed and memory reductions on T5 and RoBERTa.

19 Main Results: LLaMA**2**-**7**B—Scaling Behavior and Memory Footprint

Method	ARC	HellaSwag	MMLU	TruthfulQA	Avg.	Train Time(⇓)	Train Mem (⇓)	Inf Time(⇓)	Inf Mem(⇓)
LLaMA 2 7B	53.1	77.7	43.8	39.0	53.4	-	-	-	-
LoRA	55.6	79.3	46.9	49.9	57.9	100.0%	100.0%	100.0%	100.0%
LoRA+Prune	**46.8**	65.2	23.9	46.2	45.5	180.9%	100.0%	115.5%	68.9%
LLMPruner	39.2	67.0	24.9	40.6	42.9	**86.9%**	253.6%	**114.8%**	74.2%
APT	45.4	**71.1**	**36.9**	**46.6**	**50.0**	106.0%	**75.8%**	117.0%	**67.2%**

- At 70 percent remaining parameters, APT retains 86.4 percent performance on average.
- APT uses about 75.8 percent of LoRA training memory and remains single-GPU friendly.
- APT surpasses LoRA with post pruning and task-agnostic pruners for LLMs (large language models) at matched sparsity.

20 Ablations: Roles of Adaptive Pruning, Tuning, and Distillation

Method	SST2	MNLI	Train Time(⇓)	Train Mem(⇓)
APT	**94.5**	86.4	592.1%	70.1%
w/o $\mathcal{A}_P$	94.4	**87.5**	**82.6%**	62.2%
w/o salience	94.3	84.7	609.8%	65.0%
w/o $\mathcal{A}_T$	93.2	84.5	684.9%	64.4%
w/o $\mathcal{D}_S$	92.9	85.3	483.1%	**61.9%**

- Adaptive pruning reduces training memory and enables real inference gains.
- Adaptive tuning improves accuracy notably as sparsity increases.
- Self-distillation aids recovery with a modest training cost in time and memory.

21 Threats to Validity and Reproducibility Notes

- Hardware and kernel choices affect latency and memory numbers across devices.
- Random seeds, sparsity schedules, and rank budgets introduce variance in results.
- Strict data handling and measurement hygiene are required for fair comparisons.
- Released scripts, checkpoints, and configs support reproducibility.

22 Limitations and Future Directions

- Large language models show accuracy gaps without heavier distillation on tight budgets.
- Dynamic shape changes may require optimizer resets and careful kernel alignment.
- Richer adapter families beyond LoRA could further improve recovery.

23 Appendix: Datasets and Baselines Details (for reference)

Model	Method	Sparsity	97% TTA (s)	Train Mem. (MB)	Inf. Time (ms)	Inf. Mem (MB)
$\text{RoBERTa}_{\text{base}}$	FT	0%	127	2,696	220.8	1,157
	LoRA	0%	2,714	1,630	181.8	1,157
	LoRA+Prune	60%	6,513	1,630	84.0	869
	Prune+Distill	60%	1,899	4,544	85.2	917
	LoRA+Prune+Distill	60%	8,299	3,813	87.0	952
	APT	60%	752	1,890	91.3	904
T5_{base}	FT	0%	366	7,217	248.1	2,347
	LoRA	0%	935	4,476	254.2	2,347
	LoRA+Prune	60%	14,417	4,476	116.8	1,724
	APT	60%	1,774	5,332	185.0	1,913

- Benchmarks include GLUE, SQuAD v2, CNN or DM, and instruction tuning suites.
- Models span BERT, RoBERTa, T5, and LLaMA families across sizes.
- Baselines include LoRA with pruning, pruning with distillation, and task-agnostic pruners.

Conclusion: APT Unifies Pruning and Tuning for Efficiency

- APT jointly adapts pruning and tuning to deliver efficient training and faster inference.
- Small models retain near full accuracy with large speed and memory benefits.
- Larger models achieve strong retention under tight memory during pruning.

Thank You

Questions & Discussion